Monotonic Architectures for Lost-Sales Inventory Control

Section 1 - project objective and current compact result

Project description: replace loss-function penalties for monotonicity with architecturally enforced constraints, then test whether the resulting policy is better structured, faster to train, and more interpretable than penalty-based or unconstrained networks.

- Structure criterion: architecture-enforced candidates are PositiveWeightMLP, ConstrainedMonoMLP, MonoKAN; compliant candidates have zero raw and deployed monotonicity violations.
- Performance criterion: the best compliant architecture is PositiveWeightMLP with reward 4,525.0 per period versus PaperFCNN 3,932.5, a 15.1% gap.
- Efficiency criterion: compact training time is 2.543 seconds for MonoKAN versus 5.152 seconds for PaperFCNN, about 2.0x faster in this smoke setting.
- Interpretability criterion: MonoKAN exposes spline components; ConstrainedMonoMLP tests the C-MNN-style activation mix; PositiveWeightMLP is an expressiveness control.

Important caveat: these numbers are from compact validation artifacts. They verify the pipeline and reveal model behavior, but they are not final paper-replication evidence. The full paper_replication profile is needed for scientific comparison.

Acceptance item	Current compact result
Architecture-enforced monotonicity	Pass
Reward no worse than FCNN by 0.5%	Pass
Primary next action	Tune the best hard-monotone architecture under LS before extensions

Mathematical Formulation

Section 2 - lost-sales dynamics, base-stock structure, and diagnostics

State y_t is on-hand inventory and q_t is the purchase quantity. With no lead time and lost sales, available inventory is a_t = y_t + q_t, sales are min(d_t, a_t), and next inventory is y_{t+1} = max(a_t - d_t, 0). The one-period reward used for evaluation is revenue minus purchase and holding costs.

For a Gamma-demand product, the optimal no-lead-time base-stock policy has q*(y) = max(S - y, 0), where S is the critical-quantile order-up-to level. Therefore the desired structural derivative is dq*/dy = -1 while y < S, and dq*/dy = 0 after the cap at zero order.

Monotonicity means y_2 > y_1 implies q_theta(y_2, z) <= q_theta(y_1, z), where z contains product and demand-history features. A penalty baseline adds loss terms for violations. MonoKAN and the constrained MLPs instead restrict the function class so this inequality is enforced by the architecture.

Metric	Definition	Interpretation
Reward gap vs FCNN	100 * (R_model / R_FCNN - 1)	Performance constraint
Raw violation rate	mean[max(q(y_hi)-q(y_lo), 0) > tol]	Monotonicity before deployment caps
Deployed violation rate	same check after action cap	Operational monotonicity
Slope deviation	mean |Delta q / Delta y + 1| in active regior	Order-up-to shape
Order-up-to error	|(q(0)) - S_oracle| or grid analogue	Structural diagnostic only

Graphically, a good policy curve decreases linearly with slope -1 until it reaches zero order. A monotone curve with the wrong slope can still pass violation tests while failing the order-up-to structural test.

Operational Results

Section 3 - compact reward and service metrics

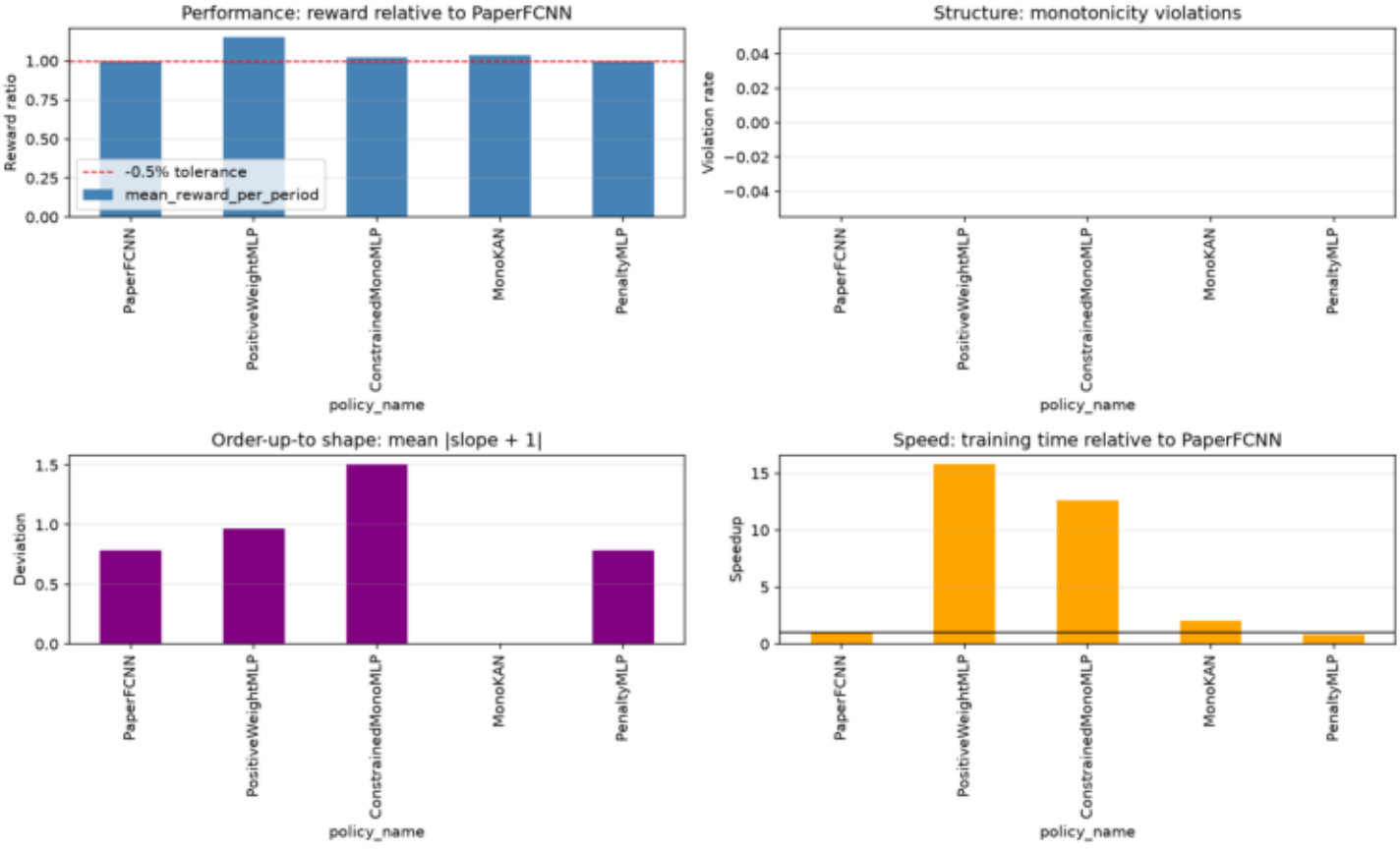
Interpretation: in the compact run, PaperFCNN and PenaltyMLP can tie when the sampled penalty is inactive. The hard-monotone architectures should be read by both zero violations and reward gap. The compact Table-3-style ranking is not paper-faithful because the product count and horizon are intentionally tiny.

Policy	Reward/period	Service	Lost sales	Mean order	Mean inventory	Train sec
PaperFCNN	3,932.5	0.835	0.165	63.784	64.256	5.152
PositiveWeightMLP	4,525.0	0.836	0.164	68.544	73.304	0.326
ConstrainedMonoMLP	4,024.7	0.752	0.248	66.140	80.970	0.409
MonoKAN	4,073.5	0.992	0.008	79.191	216.4	2.543
PenaltyMLP	3,932.5	0.835	0.165	63.784	64.256	6.405
NonOmniscientOptima	3,320.1	0.694	0.306	50.830	28.517	
OmniscientOptimal	3,522.1	0.675	0.325	52.395	30.971	
istoricalDemandBaseSt	3,330.6	0.668	0.332	50.325	27.649	

Policy	Average reward	Gap to omniscient (%)
PaperFCNN	3,932.5	11.653
PositiveWeightMLP	4,525.0	28.474
ConstrainedMonoMLP	4,024.7	14.271
MonoKAN	4,073.5	15.655
PenaltyMLP	3,932.5	11.653
NonOmniscientOptimal	3,320.1	-5.734
OmniscientOptimal	3,522.1	0.000
HistoricalDemandBaseStock	3,330.6	-5.436

Structure Dashboard

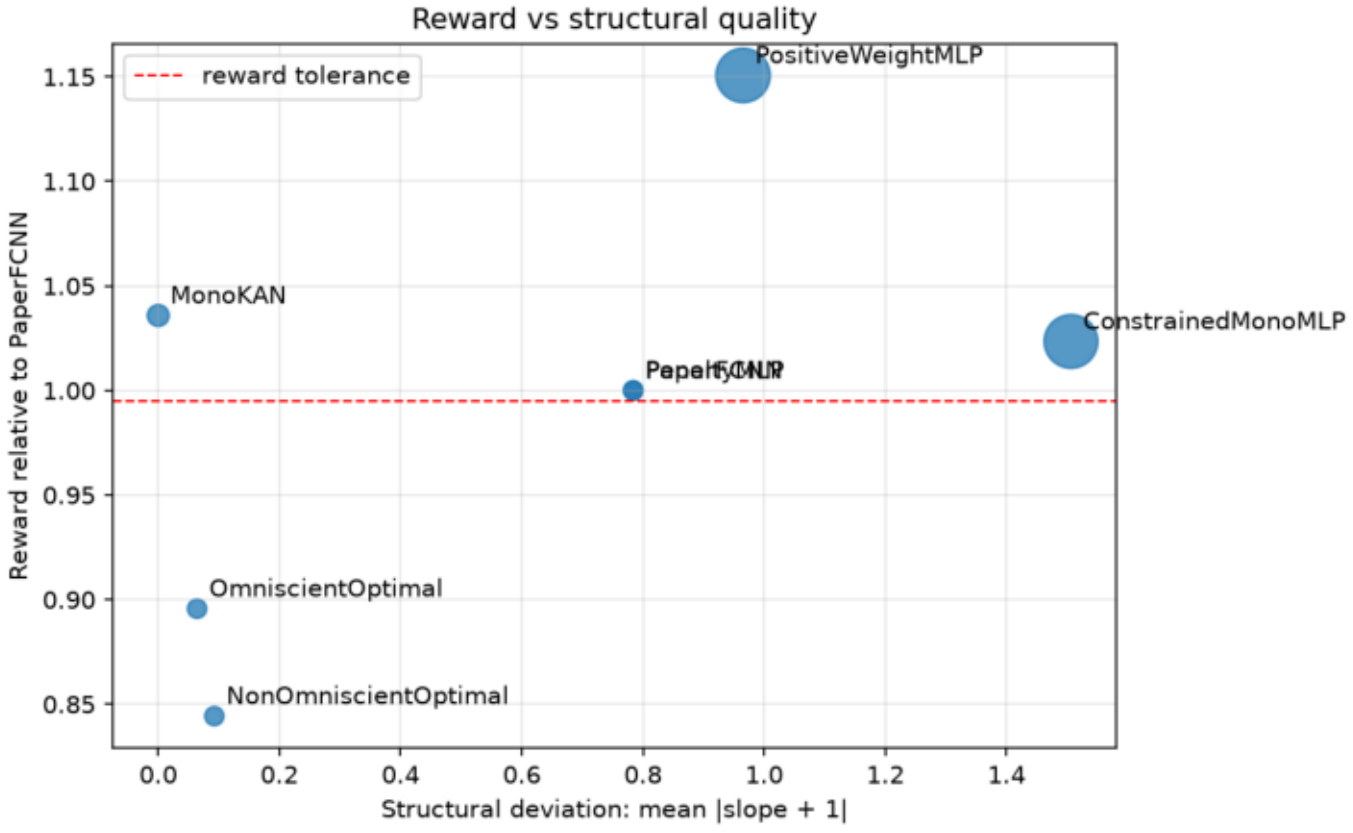
Section 4 - better, faster, interpretable criteria



Mathematical reading: the hard-monotone candidates target q(y_hi) <= q(y_lo) without penalty terms. The current best compliant architecture is PositiveWeightMLP; its reward gap is 15.1%, so the structure claim must still be paired with FCNN-level reward.

Reward-Structure Trade-off

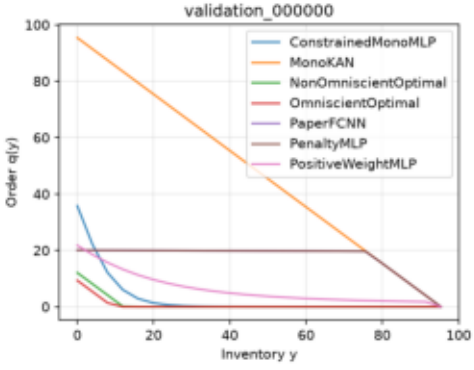
Section 5 - why the project is not just a reward benchmark

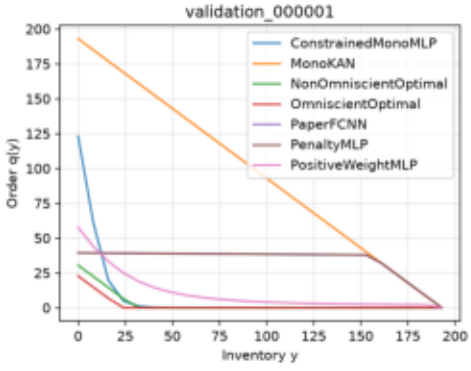


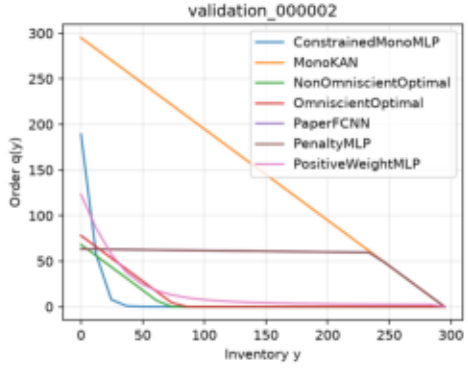
Graphical reading: the desired point is near zero violations and near zero reward gap to PaperFCNN. PenaltyMLP may sit near the FCNN performance point but does not add architecture-level guarantees. The hard-monotone points test whether exact structure can be retained without reward loss.

Policy Curves

Section 6 - purchase quantity as a function of inventory



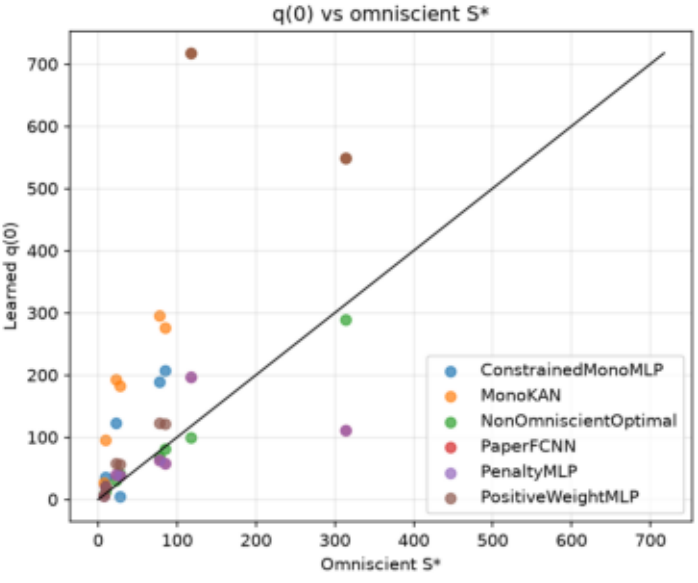


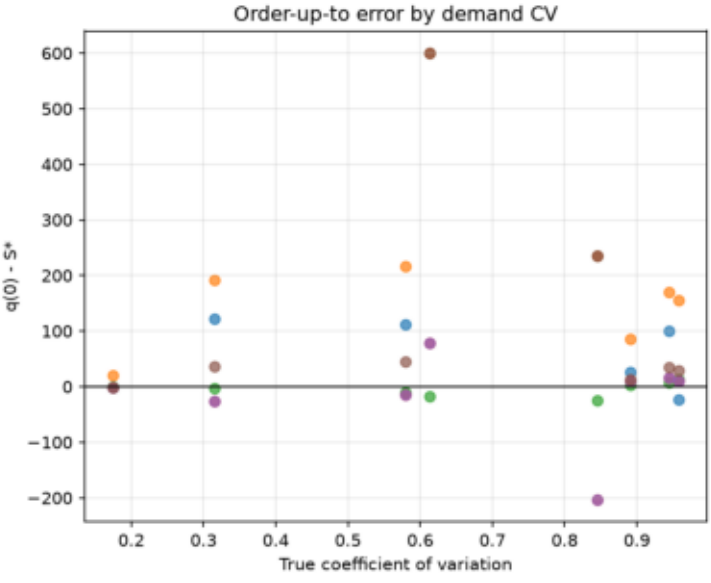


Mathematical reading: for an order-up-to policy, each curve should look like q(y)=max(S-y,0). The important visual signature is a straight line with slope -1 until it hits zero. Smoothness alone is not enough; the curve must have the correct economic slope.

Order-Up-To Diagnostics

Section 7 - structural agreement with Gamma critical quantiles

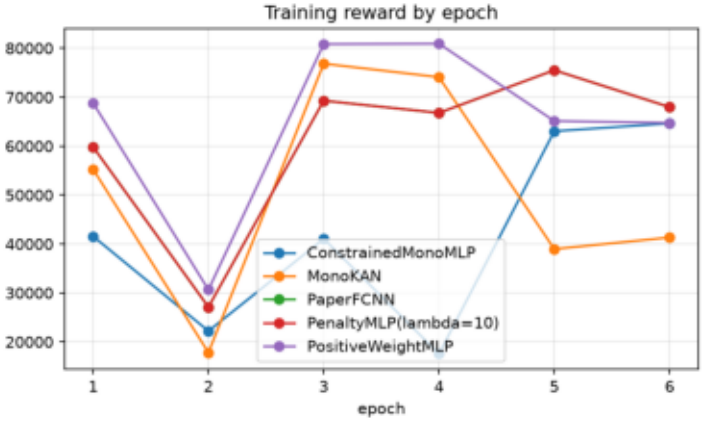


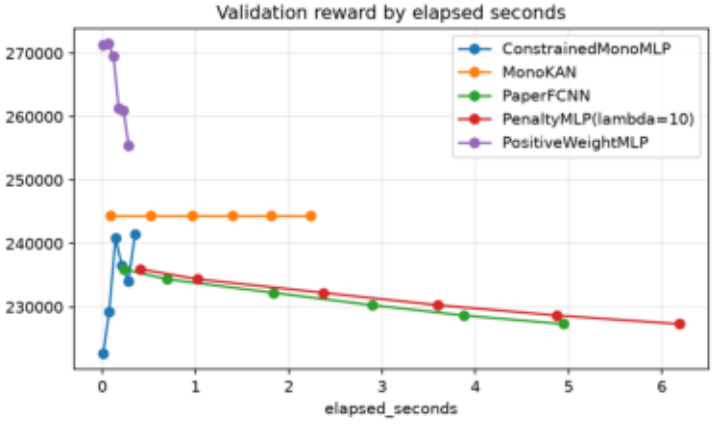


Interpretation: the oracle base-stock level S comes from the exact Gamma critical quantile. Order-up-to error is not used as a supervised training objective; it is an ex-post structural diagnostic showing whether the learned policy recovered the economic form of the solution.

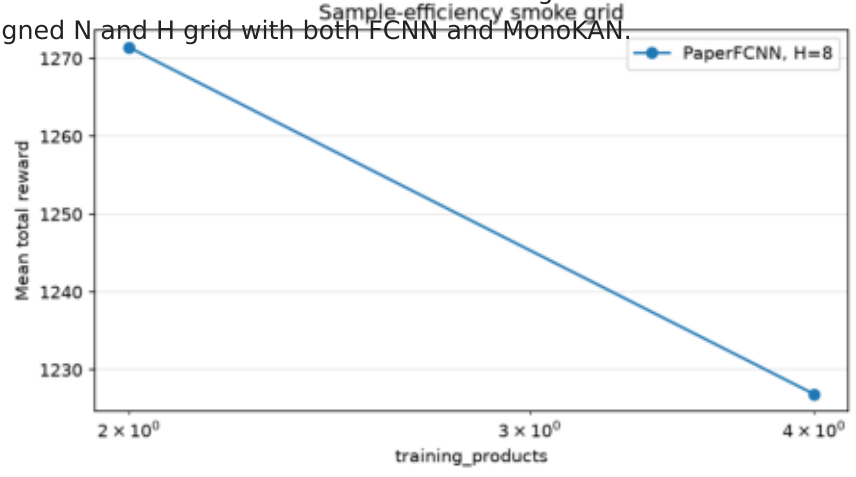
Training And Sample Efficiency

Section 8 - exploratory efficiency evidence





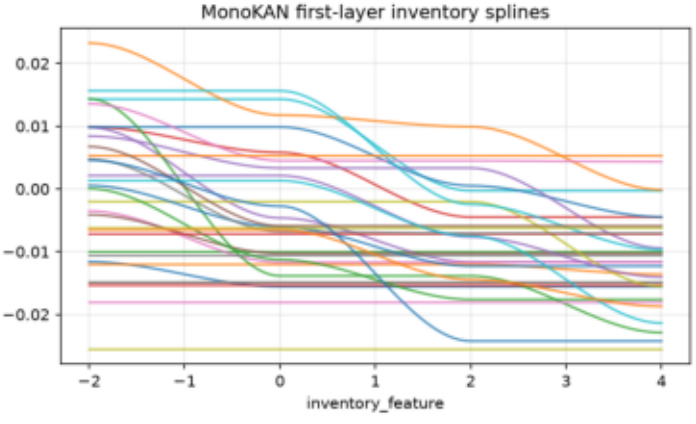
Interpretation: sample efficiency is an open research question. The current compact grid is a smoke test only, so it should not be used to claim faster learning. A credible conclusion requires the paper-aligned N and H grid with both FCNN and MonoKAN.

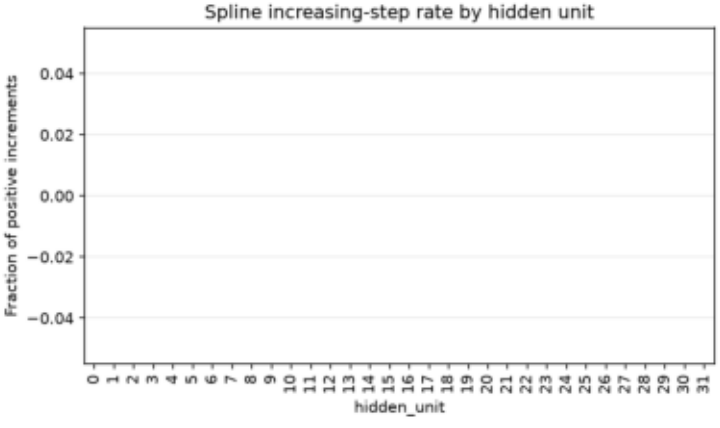


H	N	Policy	Mean total reward	Train sec
8	2	PaperFCNN	1,271.4	0.074
8	4	PaperFCNN	1,226.9	0.108

MonoKAN Spline Interpretability

Section 9 - architecture-level monotone components





Graphical reading: the first-layer inventory splines expose how the inventory coordinate enters the monotone network. This is the main interpretability difference from an unconstrained FCNN: the shape restriction can be inspected directly rather than inferred from penalty terms after training.

Conclusions And Next Steps

Section 10 - project-aligned interpretation

- The current implementation matches the project direction: LS-first, synthesized products, paper-style baselines, architecture-enforced monotonicity, penalty kept as a reference baseline.
- The compact evidence should be judged across MonoKAN, ConstrainedMonoMLP, and PositiveWeightMLP: zero raw and deployed violations are necessary but not sufficient.
- The compact evidence does not support a full project success claim unless at least one hard-monotone architecture also matches FCNN reward.
- The penalty baseline is not the proposed method. It remains useful because it mirrors the reference paper's regularization idea and demonstrates why architectural enforcement is a sharper structural intervention.

Step	Action	Reason
1	Run paper_replication LS baseline	Establish credible FCNN and optimal baselines
2	Tune hard-monotone architectures under identical	Close reward gap without adding monotonicity pen
3	Run structure-ls full validation	Produce final policy curves and violation diagnos
4	Run protected final test once	Report held-out result only after model choices are

Bottom line: the report shows mathematically and graphically whether architectural monotonicity is working, but a hard-monotone policy must still be operationally competitive with the FCNN.